

15-442/15-642: Machine Learning Systems

Data Layouts

Spring 2026

Tianqi Chen and Zhihao Jia

Carnegie Mellon University

Outline

Tiled & Thread Layout

Distributed Layout

Swizzle Layout

Outline

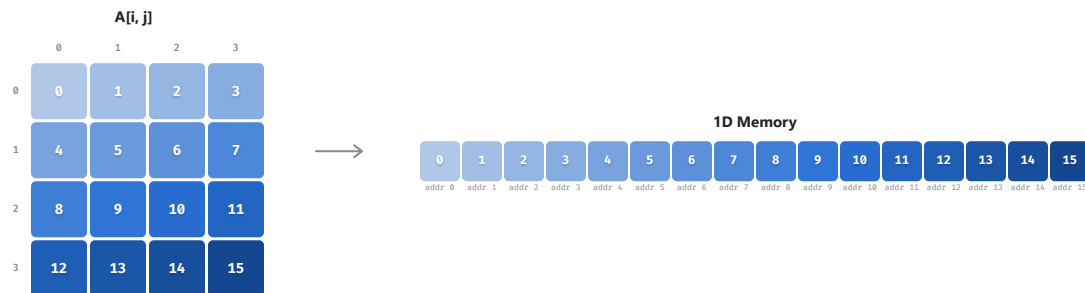
Tiled & Thread Layout

Distributed Layout

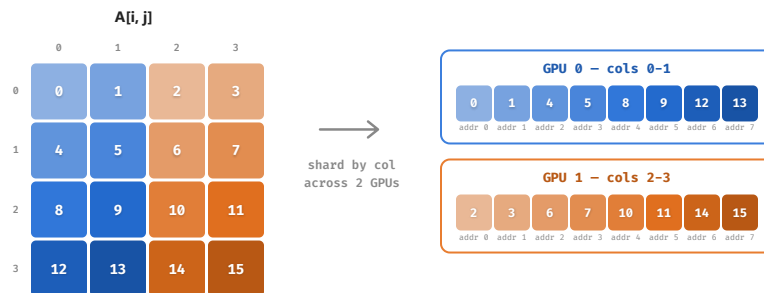
Swizzle Layout

What is Data Layout

Mapping from logical indices to the physical location of the data



or different devices...



Shape-Stride Model

We will use this notation:

$s[(4, 4) : (4, 1)]$
 shape stride

	j=0	j=1	j=2	j=3
i=0	0	1	2	3
i=1	4	5	6	7
i=2	8	9	10	11
i=3	12	13	14	15

Address computation

$$\text{addr}(i, j) = i \times 4 + j \times 1$$

$$\text{Example: } \text{addr}(2, 3) = 2 \times 4 + 3 \times 1 = 11$$

This notation can be viewed as simplified version of CuTe, with difference that we adopt row-major ordering.

Tile Layout

Representing tiling

Layout: $S[(4, 2, 2, 4) : (16, 4, 8, 1)]$

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	8	9	10	11	12	13	14	15
2	16	17	18	19	20	21	22	23
3	24	25	26	27	28	29	30	31
4	32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47
6	48	49	50	51	52	53	54	55
7	56	57	58	59	60	61	62	63

Address computation

Original shape (8 , 8)

↓ reshape

Tiled shape (4 , 2 , 2 , 4)

Mapped indices ($i//2$, $i\%2$, $j//4$, $j\%4$)

$$\text{addr} = (i//2) \times 16 + (i\%2) \times 4 + (j//4) \times 8 + (j\%4) \times 1$$

Example: $\text{addr}(2, 3) = 1 \times 16 + 0 \times 4 + 0 \times 8 + 3 \times 1 = 19$

Tile Layout: Interactive Example

S[(4, 2, 2, 4) : (16, 4, 8, 1)]

Logical view vs Physical memory | click cell to see mapping

Logical Matrix								
	c0	c1	c2	c3	c4	c5	c6	c7
r0	0	1	2	3	4	5	6	7
r1	8	9	10	11	12	13	14	15
r2	16	17	18	19	20	21	22	23
r3	24	25	26	27	28	29	30	31
r4	32	33	34	35	36	37	38	39
r5	40	41	42	43	44	45	46	47
r6	48	49	50	51	52	53	54	55
r7	56	57	58	59	60	61	62	63

<

Physical Memory (tiled 2x4)								
	+0	+1	+2	+3	+4	+5	+6	+7
0	0	1	2	3	8	9	10	11
8	4	5	6	7	12	13	14	15
16	16	17	18	19	24	25	26	27
24	20	21	22	23	28	29	30	31
32	32	33	34	35	40	41	42	43
40	36	37	38	39	44	45	46	47
48	48	49	50	51	56	57	58	59
56	52	53	54	55	60	61	62	63

<

Click a cell to see how logical element maps to physical memory

Named Axes in Strides

We need to represent different hardware axes:

- Lane and col dimension in tensor memory
- Thread indices

It is useful to have named axes, we use **@m** for normal memory

- Row-major 8×16: `S[(8, 16) : (16@m, 1@m)]`

The `@axis` tag can also be other things like `@warpid`, or `@tmemcol`

Named Axes Example: Thread + Register

Layout: S[(8, 4, 2) : (4@laneid, 1@laneid, 1@reg)]

click cell to see assignment

Logical 8x8 Matrix								
	c0	c1	c2	c3	c4	c5	c6	c7
r0	0	1	2	3	4	5	6	7
r1	8	9	10	11	12	13	14	15
r2	16	17	18	19	20	21	22	23
r3	24	25	26	27	28	29	30	31
r4	32	33	34	35	36	37	38	39
r5	40	41	42	43	44	45	46	47
r6	48	49	50	51	52	53	54	55
r7	56	57	58	59	60	61	62	63

Thread Assignment								
	c0	c1	c2	c3	c4	c5	c6	c7
r0	0	1	2	3	4	5	6	7
r1	8	9	10	11	12	13	14	15
r2	16	17	18	19	20	21	22	23
r3	24	25	26	27	28	29	30	31
r4	32	33	34	35	36	37	38	39
r5	40	41	42	43	44	45	46	47
r6	48	49	50	51	52	53	54	55
r7	56	57	58	59	60	61	62	63

Click a cell to see thread (laneid) and register assignment

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Each color pair = 1 thread (reg 0, reg 1) | laneid = threadidx % 32

Outline

Tiled & Thread Layout

Distributed Layout

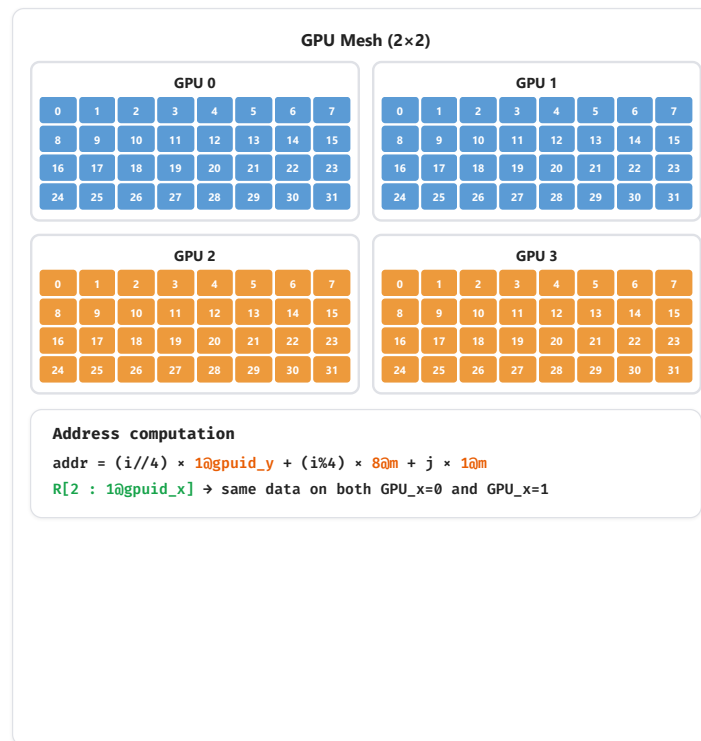
Swizzle Layout

Distributed Axes: @gpuid_y, @gpuid_x

Representing broadcast / replication

Layout: $S[(2, 4, 8) : (1@gpuid_y, 8@m, 1@m)] + R[2 : 1@gpuid_x]$

8×8 Matrix								
	c0	c1	c2	c3	c4	c5	c6	c7
r0	0	1	2	3	4	5	6	7
r1	8	9	10	11	12	13	14	15
r2	16	17	18	19	20	21	22	23
r3	24	25	26	27	28	29	30	31
r4	32	33	34	35	36	37	38	39
r5	40	41	42	43	44	45	46	47
r6	48	49	50	51	52	53	54	55
r7	56	57	58	59	60	61	62	63



Distributed Axes: Interactive

Distributed Layout — 8×8 Matrix on 2×2 GPU Mesh

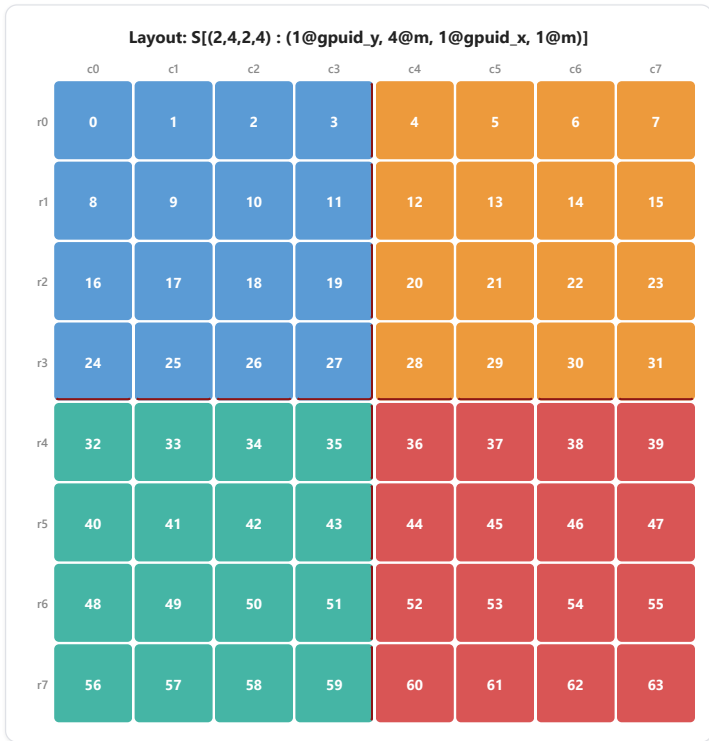
Fully sharded across 4 GPUs | click cell to see placement

Layout

Fully Sharded (S0S1)

Shard + Replica (S0R)

Shard + Offset (S0+O)



Click a cell to see GPU placement and store location

Partition 0

Partition 1

Partition 2

Partition 3

GPU cells = logical element value

Shard boundary

Mesh: GPU0=(y0,x0) GPU1=(y0,x1) GPU2=(y1,x0) GPU3=(y1,x1)

Outline

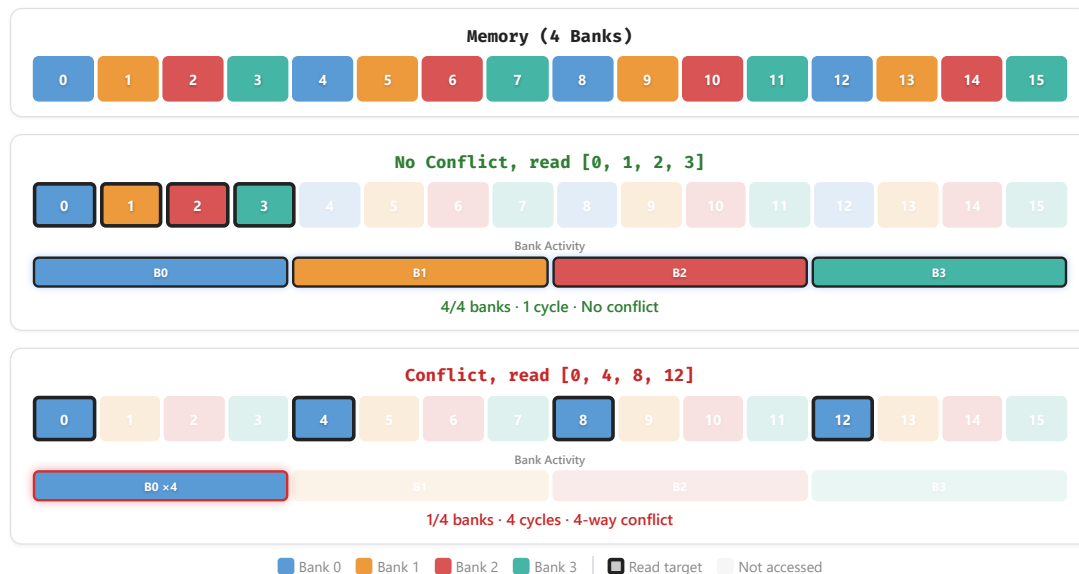
Tiled & Thread Layout

Distributed Layout

Swizzle Layout

Background: Memory Banks

Different memory banks can be read out in parallel

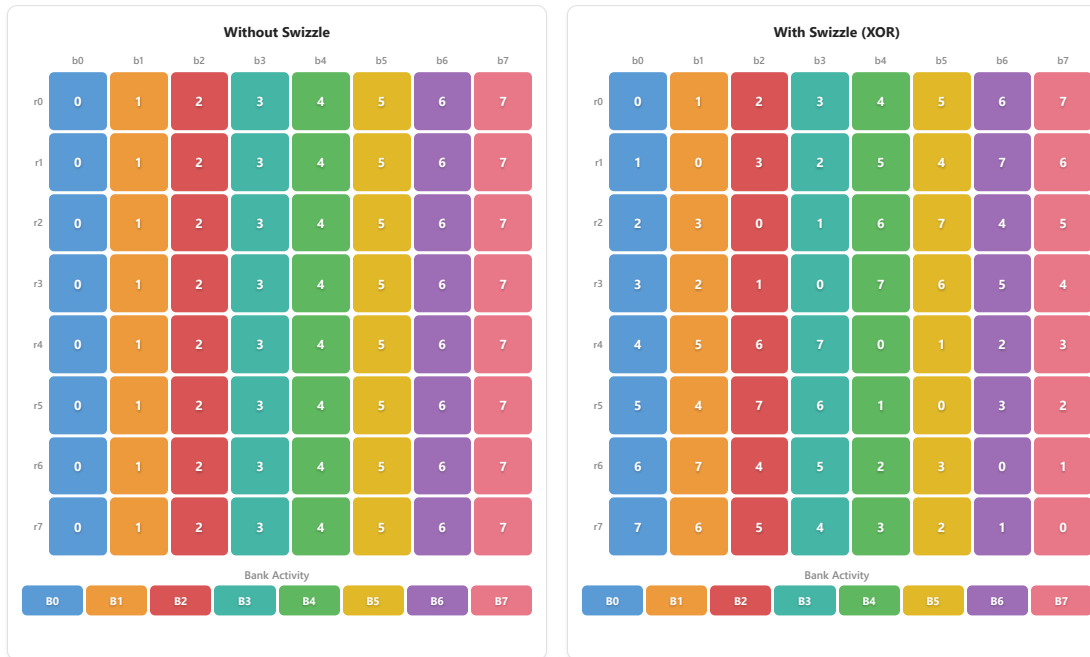


Simple Swizzle: 8×8 Interactive Example

Swizzle Layout — 8×8

mapped_col = logical_col XOR row | 8×8 matrix, 1 element = 1 bank

Read Index



☐ Bank 0 ☐ Bank 1 ☐ Bank 2 ☐ Bank 3 ☐ Bank 4 ☐ Bank 5 ☐ Bank 6 ☐ Bank 7

☐ Current read ☐ Earlier cycle ☐ Not read | Cell value = logical column | Click cell to show mapped location

Cycle-by-Cycle Reads

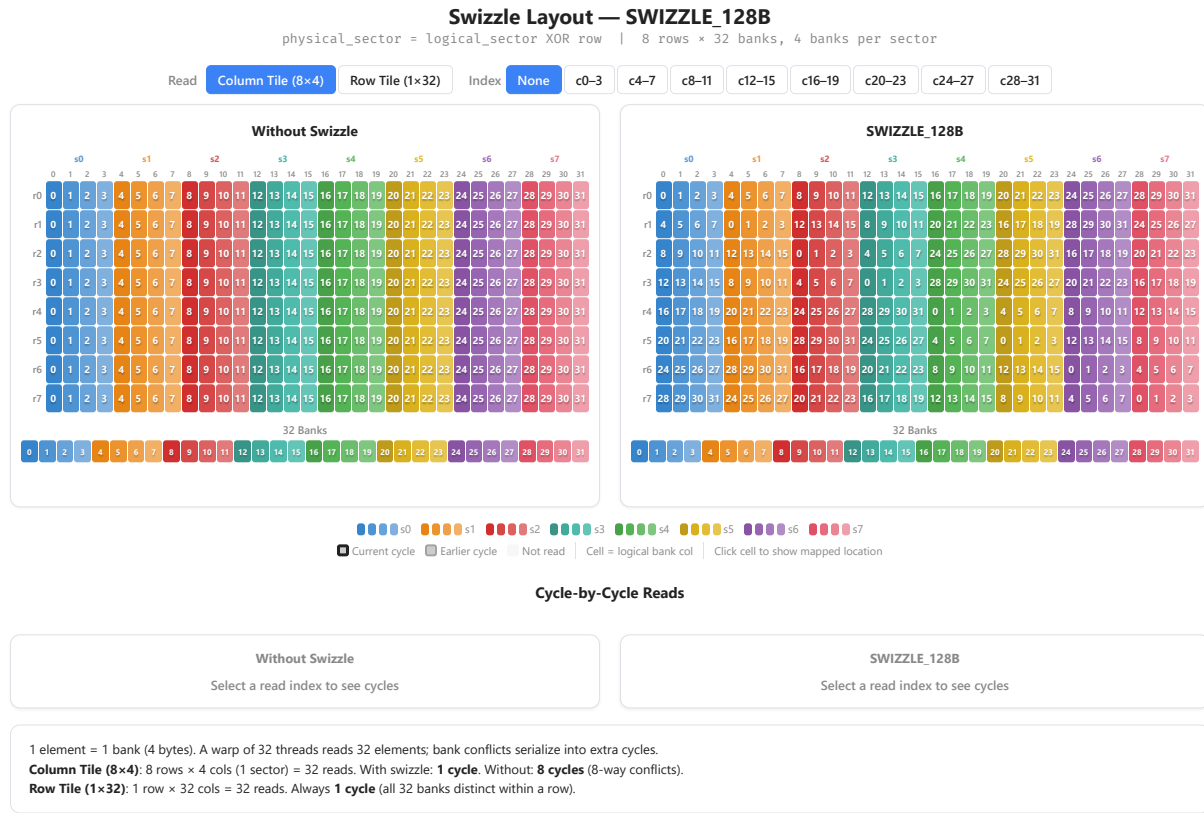
Without Swizzle

Select a read index to see cycles

With Swizzle (XOR)

Select a read index to see cycles

Real Example: SWIZZLE_128B



Swizzle as a Memory Optimization

Swizzle is an implicit remapping the hardware applies — just enable it

- Pass pre-swizzled addresses and use a consistent swizzle mode (e.g. `SWIZZLE_128B`) across ops
- Don't worry about swizzle yourself — let hardware handles it.
- Benefit: `SWIZZLE_128B` → conflict-free access to 8 rows and 8 columns at a time (fp16)

Swizzle is fundamentally a way to enable efficient column access — the idea is not specific to GPU shared memory!